

Phishing Email Classification and Analysis

Group: JIMC

Jen Kim, Inchara Kashyap, Maanya Chugh, Caitlyn Pratt

The University of Texas at Austin

I 320D: Text Mining and NLP Essentials

Dr. Abhijit Mishra

TA: Girish Karthikeyan

May 1, 2026

Introduction

Phishing attacks continue to rise, and more users are becoming vulnerable to scams that compromise their accounts or data. As everyday activities such as banking, messaging, and other services rely on online platforms, users are left with no choice but to navigate an increasingly hostile and malicious environment. One of the most common methods for these attacks is email, where they use social engineering to manipulate users' emotions and trust in order to obtain personal or financial information. These attacks can be particularly harmful for more vulnerable populations, such as the elderly, who may not have the right resources or technical support to protect themselves. Meanwhile, the sheer increase in the volume of phishing attacks adds extra pressure on security professionals, who are often faced with limited time and resources in detecting and responding to these threats. In response to this, our project aims to build models that can classify phishing emails and identify signals associated with malicious intent, enabling a more interpretable understanding of how these attacks are structured.

For this project, the dataset is imported directly from Hugging Face, with the original source from Kaggle and published by Cyber Cop under the GNU Lesser General Public License 3.0. It consists of 18,650 rows and three columns: Unnamed: 0 (row index), Email Text (email content), and Email Type (class label). To evaluate different model approaches, we implemented both a traditional machine learning pipeline and three pretrained transformer models. Our traditional pipeline combined TF-IDF and Logistic Regression for email classification and probability estimation, while the pretrained models, including T5, BERT, and RoBERTa, were used to compare performances across methodologies and provide a broader view of model behavior and interpretability.

Data Description

Exploratory Data Analysis (EDA)

The dataset contains examples for both phishing emails and safe or legitimate emails. Before preprocessing, we first examined the class distribution and found out that the dataset was moderately imbalanced, with safe emails comprising roughly 61% of the data and phishing emails accounting for about 39%. Although this imbalance was not severe, it was important to keep this in mind for the later evaluation step, as accuracy alone may not fully reflect the models' performance when the majority class is favored.

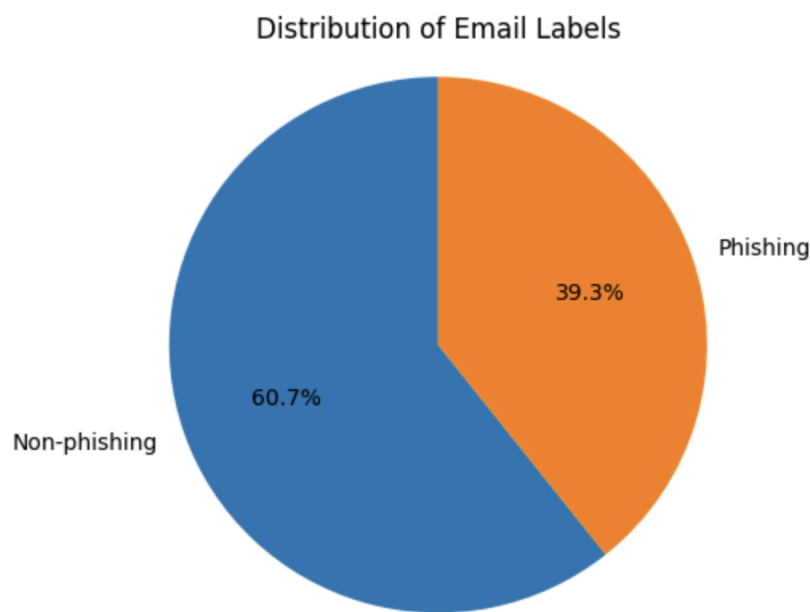


Figure 1.1. Class Distribution Before Preprocessing

Next, we compared text length between phishing and safe emails using both character count and word count. On average, safe emails were longer than phishing emails, with a mean of 3,493 characters and 686 words. However, these averages were heavily influenced by extreme outliers, where the maximum character count was 17,036,692 and the word count was 3,527,576.

This suggests that the text length itself may serve as a shortcut signal in the dataset, since unusually long emails are unlikely to reflect realistic everyday emails.

		char_count		count		mean		std		min		25%		50%		\	
Email Type																	
Phishing Email		7328.0		1610.409389		3349.372587		1.0		353.0		731.0					
Safe Email		11322.0		3493.008391		160168.024661		5.0		444.0		994.5					

		75%		max		word_count		count		mean		std		min		\	
Email Type																	
Phishing Email		1566.25		129635.0		7328.0		301.191184		569.722382		0.0					
Safe Email		2029.75		17036692.0		11322.0		685.870253		33160.236982		1.0					

		25%		50%		75%		max	
Email Type									
Phishing Email		68.0		136.0		290.0		11625.0	
Safe Email		79.0		179.0		389.0		3527576.0	

Figure 1.2. Text Length Comparison Between Safe and Phishing Emails

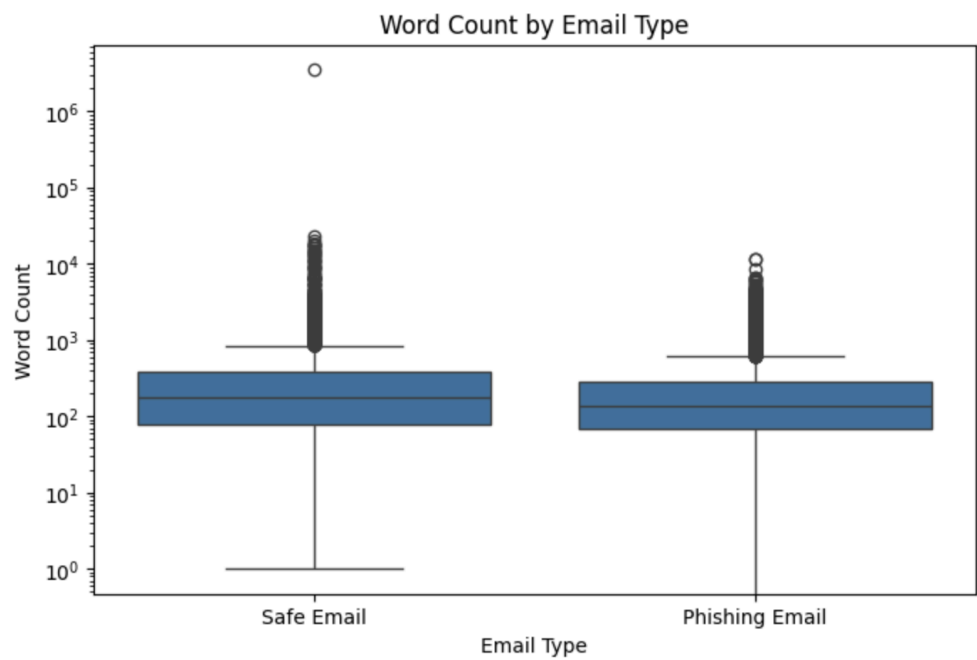


Figure 1.3. Log-scaled Boxplot Showing Word Count for Both Classes

To further analyze shared and class-specific patterns in the dataset, we examined overlapping vocabulary and repeated phrases across the two classes. For vocabulary overlap, we

calculated the Jaccard similarity score, where 1 represents identical sets and 0 represents no overlap. After removing common web and email-related cues like “http,” “www,” “com,” and “email,” the top 100 words had a Jaccard similarity score of approximately 0.24, with only 39 words overlapping. This suggests that phishing and safe emails remain relatively distinct in their most common vocabulary, which can also serve as a shortcut for models before learning deeper semantic differences in the text.

```

18 phish_tokens = []
19 safe_tokens = []
20
21 for text in phish_texts:
22     phish_tokens.extend(tokenize_overlap(text))
23
24 for text in safe_texts:
25     safe_tokens.extend(tokenize_overlap(text))
26
27 phish_counter = Counter(phish_tokens)
28 safe_counter = Counter(safe_tokens)
29
30 phish_top_100 = set(word for word, _ in phish_counter.most_common(100))
31 safe_top_100 = set(word for word, _ in safe_counter.most_common(100))
32
33 union = phish_top_100 | safe_top_100
34 overlap = phish_top_100 & safe_top_100
35 jaccard = len(overlap) / len(union) if len(union) > 0 else 0
36
37 print("Top-100 overlap count:", len(overlap))
38 print("Top-100 Jaccard overlap:", jaccard)
39 print("Overlap words:", sorted(list(overlap)))

```

Top-100 overlap count: 39
Top-100 Jaccard overlap: 0.2422360248447205
Overlap words: ['address', 'available', 'business', 'company', 'contact', 'day', 'does', 'fax', 'free', 'good', 'information', 'just', 'know', 'like',

Figure 1.4. Jaccard Similarity Calculation and Overlapping Vocabulary Between Classes

We also examined the most repeated phrases in each class to investigate if any shortcuts existed at the phrase level. In phishing emails, many frequent phrases were associated with web or HTML-related terms such as “http,” “www,” and “nbsp” (non-breaking space), along with repeating suspicious terms like “looking statements,” “spamassassin sightings,” and “investment advice.” On the other hand, safe emails consisted of repeated phrases including terms such as “etc,” “enron,” “mailing list,” “mailman listinfo,” and “original message,” which all indicate organization-specific communication and archived mailing lists. In all, these distinctive repeated phrases further support the possibility that the dataset contains shortcut signals, which the models may use to classify emails based on these patterns rather than relying on semantic understanding.

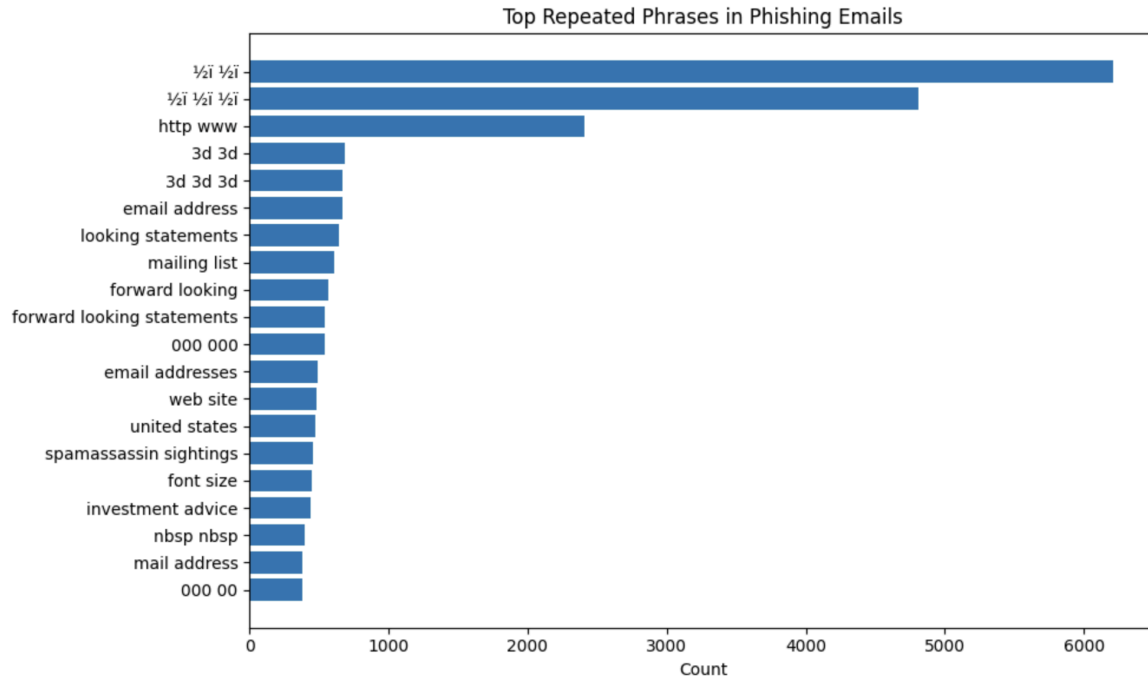


Figure 1.5. Bar Chart Showing Top Repeated Phrases in Phishing Emails

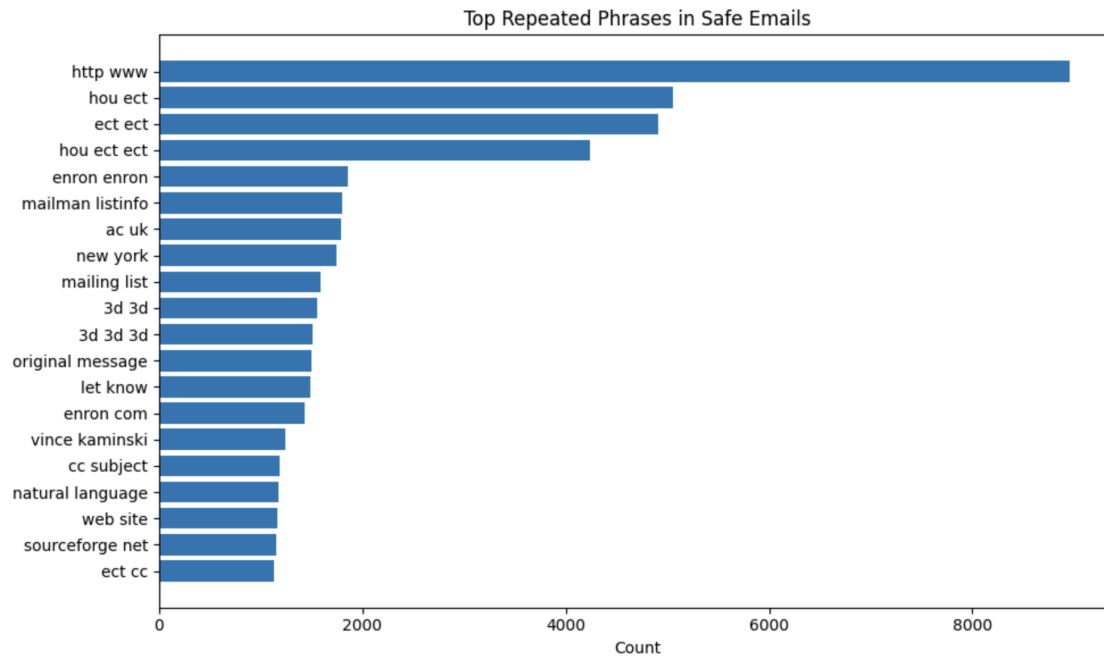


Figure 1.6. Bar Chart Showing Top Repeated Phrases in Safe Emails

Feature Extraction

For our traditional TF-IDF and Logistic Regression pipeline, we extracted four handcrafted features designed to capture surface-level signals that are commonly found in phishing emails. First, we created a small phishing keyword dictionary based on external sources (Expel & Route Fifty) listing common phishing keywords, such as “invoice,” “required,” and “verification,” which we then used to compute the total keyword count, the unique keyword count, and a binary indicator showing whether an email contained any of these terms. Next, we extracted additional structural features, including capitalization ratio, exclamation count, and URL count, that may reflect abnormal capitalization, urgency signaled through exclamation marks, or suspicious links. These handcrafted features were later combined with TF-IDF features as part of the Logistic Regression model.

Preprocessing & Splitting

For consistency and simplicity, we removed the row index column and renamed “Email Text” to “text” and “Email Type” to “label.” We then encoded the labels so that 0 represents a safe email and 1 represents a phishing email.

After our initial modeling attempt with only light preprocessing, mainly consisting of removing missing or duplicate values, which produced nearly perfect performance metrics, we revised our preprocessing strategy and shifted from optimizing pattern recognition of common phishing features to reducing potential data leakage and shortcuts. Therefore, our standardized preprocessing step applied more aggressive normalization techniques across all models, including lowercasing, removing URLs and punctuation, and normalizing whitespace. In addition, identifiers such as email addresses, numbers, and dates were replaced with placeholders so that the models would not memorize specific tokens or strings when making predictions. We

also removed empty and duplicate values and standardized text length by filtering out emails longer than 500 words or shorter than 20 words, especially given the significantly higher average email length for safe emails, as previously mentioned. This preprocessing method thereby reduces the likelihood that the models primarily rely on dataset-specific patterns or are affected by data leakage.

Following preprocessing, the dataset was split into an 80/10/10 ratio, with 80% for training, 10% for validation, and 10% for testing. We also checked for overlap across the three subsets and found none, suggesting no data leakage between the train, validation, and test sets.

Methodology

Model Selection

For the traditional pipeline, TF-IDF was chosen since it emphasizes document-specific terms while penalizing words that tend to appear frequently across all text, making this technique applicable for differentiating between class-specific patterns in the text. We captured keywords and phrases that are commonly associated with phishing language. Along with TF-IDF vectors, we incorporated 6 handcrafted features: total and unique phishing keyword count, a binary keyword indicator, capitalization ratio, exclamation count, and URL count. These features capture the structural phishing signals that are frequently weaponized. Logistic regression was selected since it produces a classification probability, adding to interpretability. The coefficients of the model can be ranked to show which words and features are more influential in the model's predictions for each class. A limitation of this approach however is that TF-IDF is a bag-of-words model which discards word order and contextual relationships. Therefore, the model may rely on dataset-specific vocabulary rather than generalizable phishing patterns.

For pretrained models, BERT and RoBERTa were chosen since their bidirectional architecture is applicable to classification tasks. The model takes in the full context of a text in both directions. RoBERTa was a key method since it is more robust than BERT, as it underwent robust pretraining on a larger and more diverse corpus. In addition, T5, which is also robustly trained on a mixed-domain corpus, was chosen to comparatively evaluate a text-to-text generative task to an encoder-based classification.

A common limitation across the three transformer models is that they were pretrained mostly on English corpora, which led to some misclassifications on non-English email texts in the dataset. The fine-tuning process, which is described in detail below, was computationally expensive and required GPU resources.

Fine-Tuning

For fine-tuning the transfer-learning models, we set the seed and used the same preprocessing techniques to ensure reproducible results. The rows in the training and validation dataset were transferred into numerical representations using the respective models tokenizer, and the text lengths were cut to 256 tokens based on the median and mean values found for word count. The training arguments set the model to train over three full passes of the dataset, or three epochs. Additionally, the batch size was set to 8, indicating the weights are updated every 8 samples. Lastly, the learning rate was set to $2e-5$ so the weights would update in a stable manner. These argument settings contribute to the transformer models learning the intricate semantic patterns in the training data to create a well fitted model. The validation dataset was used during training to evaluate the model at each logged step.

Results

Comparative Classification Results

Before comparing all models, we briefly evaluated whether adding handcrafted features to the TF-IDF and Logistic Regression model improved classification performance. These features, as previously mentioned, are designed to capture keyword frequency, capitalization ratio, exclamation count, and URL count. However, adding these features to the model's feature set did not produce a meaningful improvement in the model's baseline. This is likely because the lexical representation already captured most of the distinctive information needed to classify emails, in which the handcrafted features contributed nothing much beyond the word patterns.

Accuracy: 0.9576407506702412					
	precision	recall	f1-score	support	
0	0.98	0.95	0.96	1132	
1	0.93	0.96	0.95	733	
accuracy			0.96	1865	
macro avg	0.95	0.96	0.96	1865	
weighted avg	0.96	0.96	0.96	1865	

Figure 2.1. Classification Report for TF-IDF + Logistic Regression Model without Features

Accuracy: 0.9581769436997319					
	precision	recall	f1-score	support	
0	0.98	0.95	0.97	1132	
1	0.93	0.97	0.95	733	
accuracy			0.96	1865	
macro avg	0.95	0.96	0.96	1865	
weighted avg	0.96	0.96	0.96	1865	

Figure 2.2. Classification Report for TF-IDF + Logistic Regression Model with Features

Our evaluation metrics include accuracy, precision, recall, and F1-score. Accuracy provides an overall measure of model performance, but because the primary goal of this project

is detecting phishing emails, we focused on class 1 metrics. In particular, recall is especially important because it measures how well a model identifies actual phishing emails without missing them. In a real-world setting, false negatives are costly because they represent malicious emails that go undetected, which can result in further harm for stakeholders. Precision is also important, as it reflects how reliable a model’s phishing predictions are. Therefore, these metrics were compared across models using classification reports and confusion matrices.

Overall, all four models performed very strongly, with the transformer-based models having the highest scores. The TF-IDF with Logistic Regression model with handcrafted features achieved an accuracy of about 0.985 and a macro-average F1-score of 0.98. This model performed relatively strongly among the others, even though it is a bag-of-words model relying on handcrafted features. This indicates that the dataset contains strong language patterns that are decipherable without using deep context understanding.

TF-IDF + LR w/ features					
Accuracy: 0.9850533807829182					
	precision	recall	f1-score	support	
0	0.99	0.99	0.99	875	
1	0.98	0.98	0.98	530	
accuracy			0.99	1405	
macro avg	0.98	0.98	0.98	1405	
weighted avg	0.99	0.99	0.99	1405	

Figure 2.3. Classification Report for TF-IDF + LR Model with Handcrafted Features

BERT in particular achieved the strongest overall performance out of all models, achieving an accuracy of about 0.99 with a macro-average F1-score of 0.99. This model achieved a perfect precision for the phishing class and a near-perfect recall of 0.99 on the

phishing class, indicating almost no false positives while capturing almost all phishing emails. BERT results show that it is the most balanced model overall.

BERT				
Accuracy: 0.993594306049822				
	precision	recall	f1-score	support
0	0.99	1.00	0.99	875
1	1.00	0.99	0.99	530
accuracy			0.99	1405
macro avg	0.99	0.99	0.99	1405
weighted avg	0.99	0.99	0.99	1405

Figure 2.4. Classification Report for BERT

Similarly, RoBERTa achieved almost as strongly, with an accuracy of about 0.993 with a macro-average F1-score of 0.99. Precision was perfect for the phishing class, along with a recall of 0.98. This suggests that both transformer-based encoder models are well-suited for phishing classification, with BERT performing slightly better in terms of recall for the phishing class.

RoBERTa				
Accuracy: 0.9928825622775801				
	precision	recall	f1-score	support
0	0.99	1.00	0.99	875
1	1.00	0.98	0.99	530
accuracy			0.99	1405
macro avg	0.99	0.99	0.99	1405
weighted avg	0.99	0.99	0.99	1405

Figure 2.5. Classification Report for RoBERTa

Lastly, T5 performed the weakest out of all four models implemented, with an accuracy of about 0.945 and macro-average F1-score of 0.94, as well as recall of 0.88 and precision of 0.98 for the phishing class. Although its precision is very high, the lower recall suggests that the

model missed more phishing emails than the other models, making it the least desirable model to implement for phishing classification.

T5				
Accuracy: 0.9451957295373665				
	precision	recall	f1-score	support
0	0.93	0.99	0.96	875
1	0.98	0.88	0.92	530
accuracy			0.95	1405
macro avg	0.95	0.93	0.94	1405
weighted avg	0.95	0.95	0.94	1405

Figure 2.6. Classification Report for T5

Model Performance Interpretation and Key Findings

The confusion matrices below provide a visual of each model's errors across both classes. TF-IDF with logistic regression produced 8 false positives and 13 false negatives. BERT and RoBERTa had very similar error distributions, both producing 2 false positives, and BERT producing 7 false negatives while RoBERTa produced 8. T5 had the most errors across all methods, producing 11 false positives and 66 false negatives. Across all methods, false negatives were more common than false positives. However, this is a more harmful error type since phishing emails incorrectly classified as safe can cause more security risks for especially vulnerable users such as the elderly. BERT's high scores make it the most well-balanced and applicable to real-world contexts compared to other models. T5's high number of false negatives indicates a major limitation, making it least applicable to real-world contexts. T5's weak performance could be attributed to its text-to-text generative framing which includes an added layer of complexity during fine-tuning.

The overall high performance across all models indicates that the dataset is able to be relatively separated based on distinct vocabulary distributions. However, the high accuracy across the models may be a result of the dataset being easily separable by domain type instead of the models truly understanding differences across real phishing and safe emails and learning generalizable phishing semantics that are applicable to real-world contexts of email data.

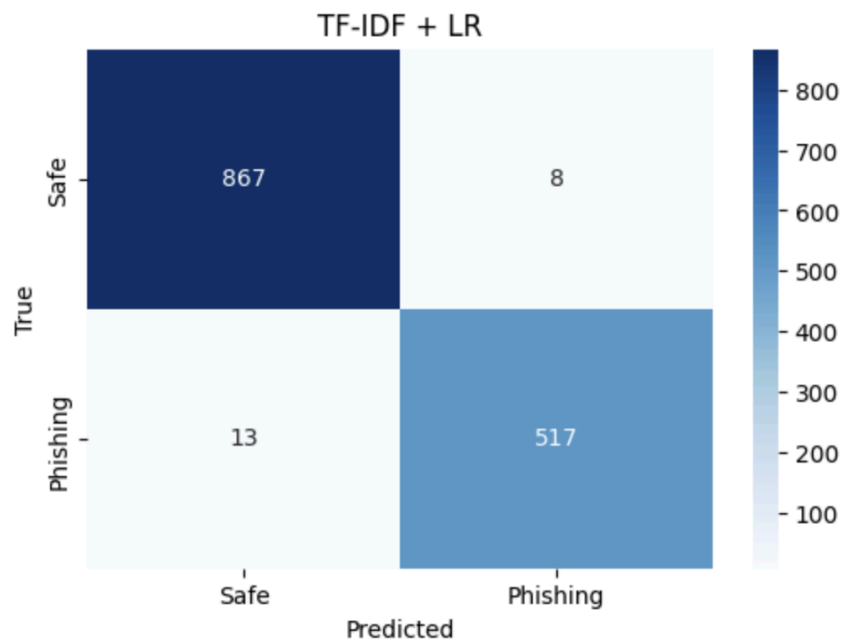


Figure 3.1. TF-IDF + LR Model Confusion Matrix

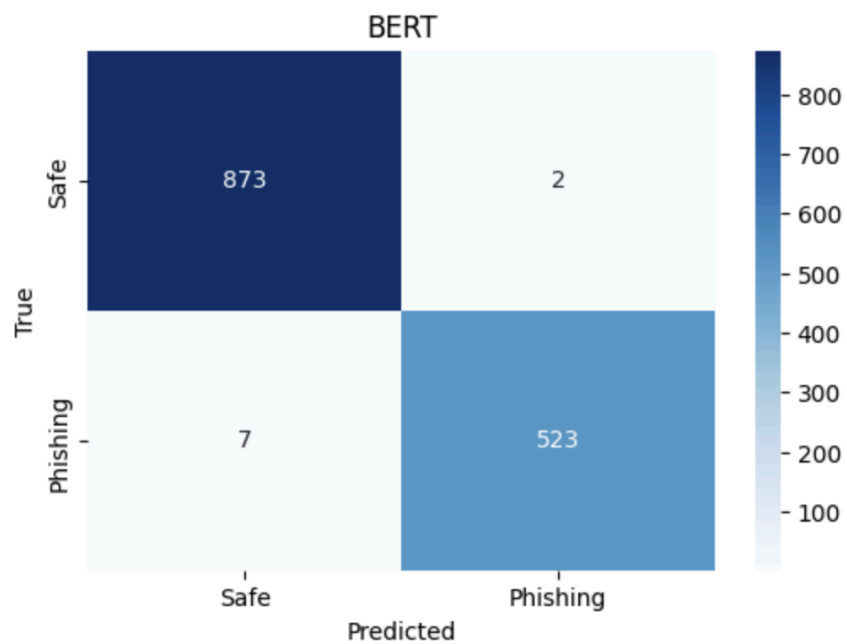


Figure 3.2. BERT Confusion Matrix

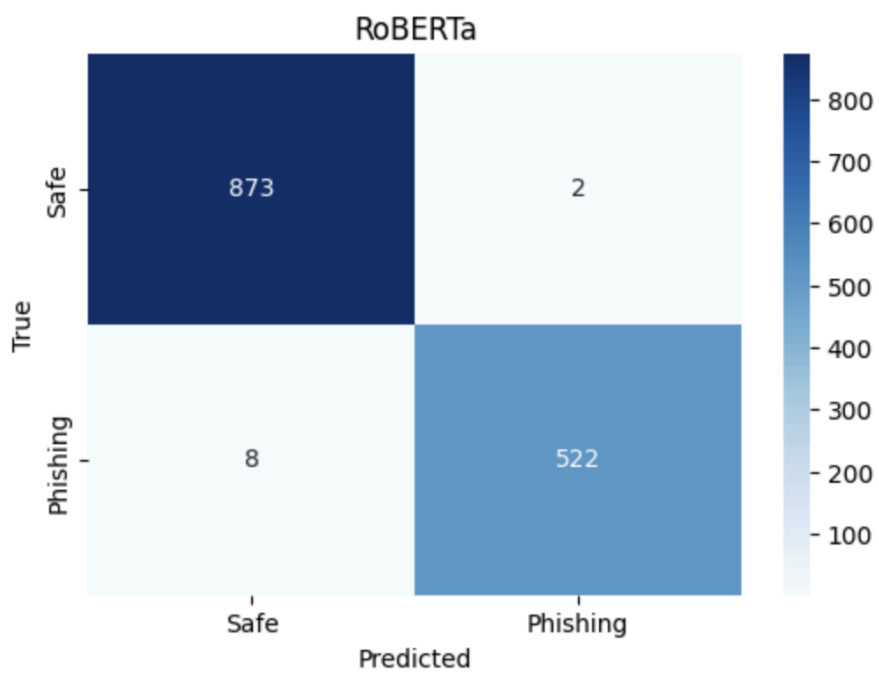


Figure 3.3. RoBERTa Confusion Matrix

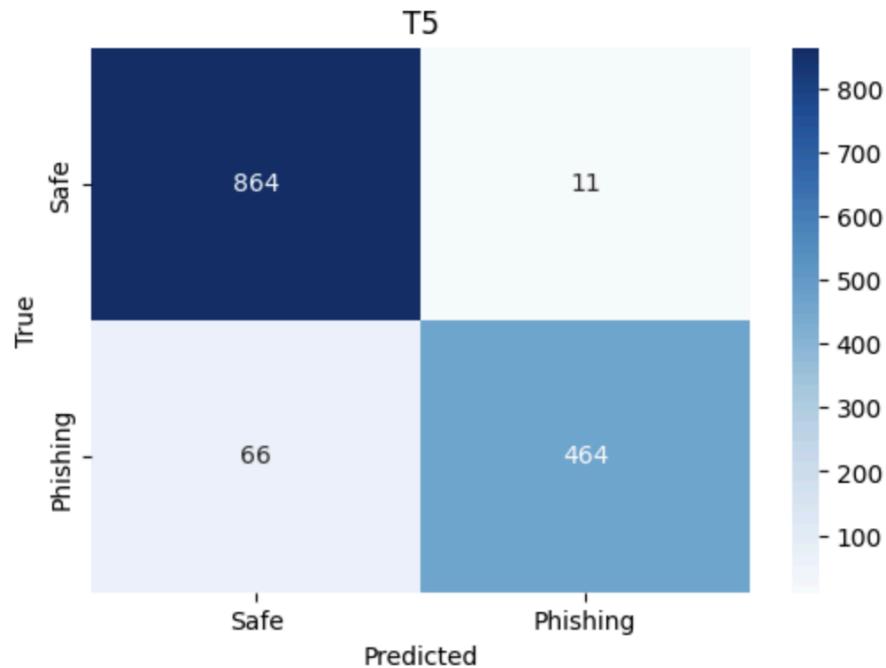


Figure 3.4. T5 Confusion Matrix

Interpretability: LDA Topic Modeling

Our team implemented Latent Dirichlet Allocation (LDA), a topic modeling technique, to extract the major themes and differences between phishing emails and safe emails. The LDA method takes in the TF-IDF vector word representations, and estimates the probability of a word in a topic and a topic in a document using a directed graphical model, or Bayesian network. We analyzed the words with the top relevancy scores and created word clouds to form email topics for both fake and real emails.

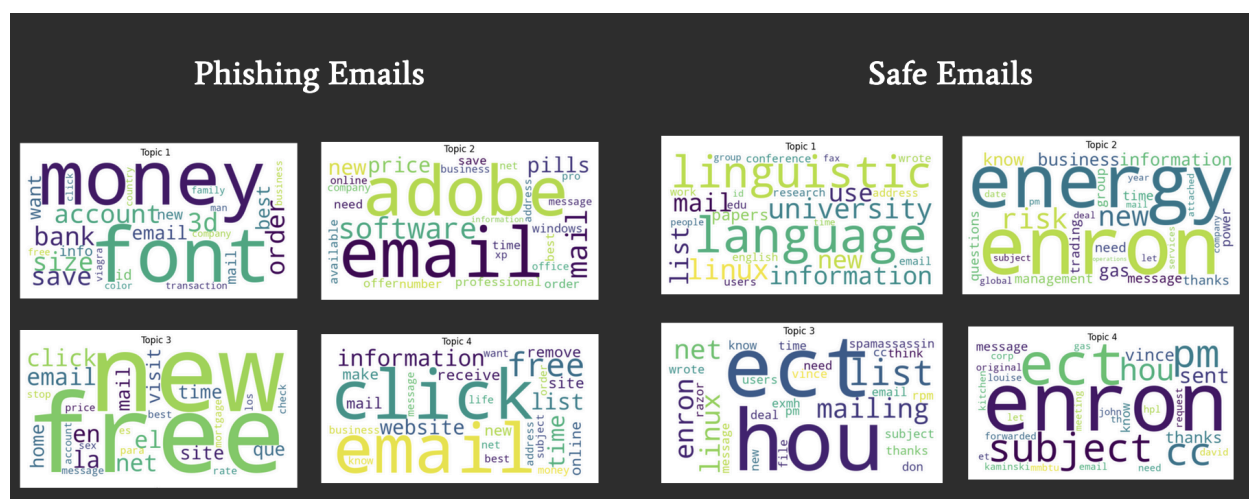


Figure 4.1. LDA Topics for Both Classes

Our team determined four major topics in the documents based on human evaluation of group separation for different topic numbers. For safe emails, the first group appeared to be academic communities with words like “university,” “information,” and “research.” The second group appeared to be energy- based business communication with the words “enron,” “energy,” and “gas.” The third group had the words “linux” and “spamassassin,” suggesting the emails originated from technical communities. And lastly, the fourth group appeared to have standard email language, with words like “subject,” “cc,” and “thanks.”

For phishing emails, the first topic appeared to be business scams with words like “money,” “account,” and “bank.” The second topic was software and product scams with the words “pills,” “adobe,” and “software,” and the third topic appeared to have attention-grabbing words like “new,” “free,” and “visit.” The last topic appeared to be mass messaging language with the words “click,” “receive,” “email,” and “website.” The topics suggest that the safe emails in the dataset may be potentially curated from universities, energy companies, and open-source communities, whereas phishing emails contained bank impersonation, product offers, and online

scams. The clear distinction in email subjects suggest that the dataset may be easily separable based on the email theme and included tokens.

A limitation of the topic modeling method is the inability to recognize structure, sequence, or grammar due to its unsupervised nature for categorizing individual words. This causes the model to struggle with creative texts, like sarcasm or colloquial terms, that are common in human communication. Lastly, the model is limited by the need to be evaluated by people, which may lead to subjective interpretations.

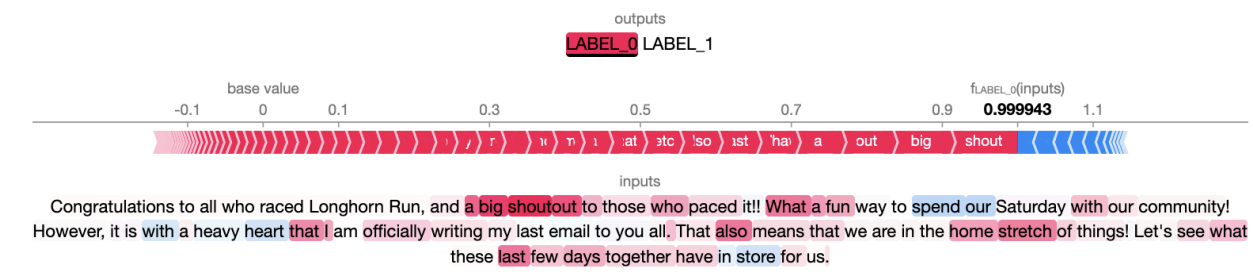
Interpretability: TF-IDF + Logistic Regression

Top phishing email features:			Top safe email features:		
	feature	coefficient		feature	coefficient
9903	your	5.840430	2501	enron	-6.216857
9805	you	4.941171	9755	wrote	-3.797446
3505	here	4.182527	8103	the	-3.709178
3649	http	4.098833	9135	url	-3.641455
6021	our	4.079967	5820	on	-3.602498
5326	no	3.340959	9136	url date	-3.559723
1504	click	3.009878	8047	that	-3.501657
1506	click here	2.922327	8038	thanks	-3.414653
5088	money	2.848024	1946	date num	-2.999185
6912	remove	2.676535	1937	date	-2.921680
3043	free	2.623950	4387	language	-2.800814
4503	life	2.444196	9294	vince	-2.691084
9276	viagra	2.424891	9084	university	-2.488318
3907	info	2.415080	4541	linguistics	-2.482507
7551	software	2.381202	2385	edu	-2.276267
7484	site	2.293613	755	attached	-2.244286
4689	low	2.254677	4567	list	-2.222281
7185	save	2.239728	2506	enron com	-2.200112
5893	online	2.192181	4680	louise	-2.194437
6076	over num	2.125845	8678	to	-2.030256

Figure 4.2. Sorted Lists of TF-IDF Word-Coefficient Pairs for Both Classes

For our next interpretability method, we used feature and coefficient extraction from the TF-IDF and Logistic Regression model to find the unigrams and bigrams that contributed most towards the model's predictions. Words like “you,” “click here,” “money,” and “free” contributed

heavily towards the phishing label. These words, especially with the words “you” and “your” having the heaviest coefficients, show that the phishing emails are attention-grabbing and intend to make the user take action. In safe emails, the company name “enron” had the strongest coefficient that contributed towards the safe label, further proving that the model was able to easily separate the labels based on the perceived email source and subject type. Other words in the safe category like "language" and “university” show other common subject types within the safe category for the dataset, but the safety of the email being determined by these subject types and recognized names suggests our dataset may be ill fitted to represent all real emails.



To further evaluate how the models classify emails, we applied SHAP on the RoBERTa model to interpret and visualize how different words and phrases contributed towards the transfer model's predictions, along with the confidence score. The email used was a real email received about the Longhorn Run from the Running Club at the University of Texas. As shown above, the tokens "spend" and "store" slightly contributed towards the phishing label. The model may have categorized these words as money related terms that are typically used in phishing language, misinterpreting how the words are intended within the context of the sentence. However, a majority of the terms, including "big shout out," "home stretch," and "last few days together"

contributed to the safe label (Label_0). The model's recognition and emphasis on community focused phrases over popular tokens shows the influence of the model's deeper complexity. RoBERTa categorized the friendly language in the email as safe, suggesting safe emails in the dataset tend to contain similar tones of encouragement and personability.

Conclusion

Each step of our model building pipeline produced insights into phishing and safe email patterns in our dataset. Through preprocessing, we found phishing emails tend to have shorter word count on average compared to safe emails. During feature extraction and testing, we found the crafted features had little effect on our models accuracy results, contributing to the idea that the data is easily separable by confounding factors. The low overlap in Jaccard similarity of phrases suggests different tokens are used between the classes. Through our results, we found all models were able to easily classify emails, with BERT performing the best. In topic modeling, we found distinct patterns in topics that give insight into the class separability. Safe emails had subjects that included mainly energy-based business, universities, open source communities, and used typical emailing language, whereas phishing emails had themes of product and software scams, bank account fraud, online scams, and used mass-messaging, generic language, further cementing the idea the emails are easily classified by included tokens. Our interpretability methods from the TF-IDF and Logistic Regression model prove that the tokens used in phishing emails are typically attention-grabbing, and attempt to influence the recipient using largely scalable words. The interpretability method from the more complex transformer model found friendly language and community-focused phrases to be typical communication patterns in safe emails. The key differences in classes demonstrated throughout our project suggest the text in the dataset can easily be classified by the tokens that indicate the domain and subject of the email.

Although our results indicate high performing models, the outcome is mainly due to dataset-specific patterns, like email source or subject. This causes poor generalizability of our findings, creating limitations in our insights. Our comparisons for testing the extraction of common phishing features and differing model complexity on classification reports were largely obsolete, due to all variations resulting in high accuracy. Further research could be conducted testing the created models on outside emails that do not contain the specific domains identified in this dataset to discover real-world accuracy, generalizability, and scalability. Furthermore, similar methods should be conducted using differing email datasets to test model complexities and common phishing features, including retaining punctuation and original capitalization, to find whether these features are true indicators of phishing emails.

Our transfer learning models are further limited due to the lack of diversity in the mainly english text-based dataset the models were trained on, and the mainly english corpus the transformer models were pretrained on. Real phishing attacks include various languages, image-based phishing, and modern adversarial prompts curated to bypass phishing detectors, but these cases were not included in our pipeline. Future research may include utilizing transformer models pretrained on multiple languages and datasets with emails in various languages to promote model generalizability and scalability. Additionally, a dataset with a diversity of attack types, including images and adversarial prompts, may be considered in future work.

Lastly, the class imbalance with a lower number of phishing emails may have contributed to the results producing more false negatives and creating lower recall scores. This causes phishing emails to be incorrectly labeled safe, potentially producing harm to users and lowering user trust. Similarly, a large amount of false positives that filter out important, safe emails may reduce trust and usability. Both cases need to be avoided, so the ideal dataset would have an even

split between classes. Although our best model achieved a 99% recall score and 100% for precision, future research should consider a fair class distribution in the dataset make-up to avoid outcome biases.

Appendix

Q: What dataset was used, and how representative is it of modern phishing campaigns?

A: A phishing dataset from Kaggle published under the GNU Lesser General Public License 3.0 was used for this project, containing examples labeled as either phishing or safe. It is not fully representative of modern phishing campaigns, which now often rely on more sophisticated social engineering tactics, such as impersonation, contextual personalization, and malicious links or attachments disguised as legitimate documents, QR codes, and more. Therefore, phishing detection can no longer rely only on traditional surface-level cues such as typos, unusual capitalization, or punctuation errors due to a more polished and convincing manner. This leads us to conclude that although the dataset used in this project is useful for benchmarking and model comparison, it cannot fully represent real-life modern phishing threats.

Q: How do you counter class imbalance during training and evaluation?

A: The path from class imbalance to outcome bias is a prominent issue in machine learning. A majority of peer reviewed articles hesitate to give a clear answer, and the survey used, Leevy et al. 2018, discloses a need for deeper comparative studies for the issue. For data-level methods to address class imbalance, the survey suggests the Synthetic Minority Over-Sampling Technique (SMOTE). The method involves forming new minority class instances by interpolating values between existing minority class instances. A concern for the implementation of this method is the model overfitting to the dataset, leading to poor generalizability to real world data. In the

context of our dataset, the issue of overfitting is already a prominent concern causing this method to be unsuitable for our dataset. However, this method may be effective in bigger datasets that already have greater diversity within class architectures. For algorithm-level methods, the cost sensitive method could be implemented by assigning more weight to an instance or learner in the event of a misclassification. For our model, instances where the model incorrectly predicts a phishing email to be safe or a false negative would cause the greatest punishment during model training. Although this method does not directly alter the class imbalance, the technique has potential to prevent model bias in outcome predictions.

Q: Any specific indicators like words that signaled phishing emails?

A: As demonstrated with our traditional model, the features we extracted didn't have much influence on the model's performance, including keywords, exclamations, capitalizations, and URLs. Our LDA topic modeling showed that there is a noticeable difference between the sources, where phishing emails come from web-based/scam-generated emails, while safe emails come from legitimate entities, represented by the specific domain names, such as edu, enron, and university.

Q: How did your models explain their decisions, especially for borderline phishing emails?

A: For borderline phishing emails, the models explained their decisions using both feature-based and model-agnostic interpretability methods. For the TF-IDF model, we analyzed feature coefficients to identify which words and phrases most strongly influenced predictions. Tokens such as "click," "free," and "money" pushed predictions toward the phishing class, while terms like "enron" and "university" were associated with safe emails.

For the transformer-based models (BERT, RoBERTa, and T5), we used SHAP to understand how individual tokens contributed to predictions. SHAP highlighted how contextual phrases, tone, and wording influenced the model's output rather than just isolated keywords. For example, friendly or community oriented language increased the likelihood of safe classification, while urgency and transactional language contributed to phishing predictions.

In borderline cases, these explanations revealed a key limitation: models sometimes relied too heavily on specific keywords and patterns, even when the overall context suggested otherwise. This led to occasional misclassifications, especially when legitimate emails contained terms commonly associated with phishing, like "account" and "payment," or when phishing emails were written in a more casual and natural tone.

Q: How would performance change on real corporate inbox data with unseen phishing templates?

A: Further testing would need to be conducted to truly evaluate performance on real corporate emails. Since our models mainly learned dataset specific patterns, performance would depend on the corporate domain. For example, real banking emails may be mislabeled as phishing attacks due to the examples seen in the training data. In contrast, university, open source, and energy company based emails may be correctly identified as safe. Unseen templates for phishing may result in lower confidence scores and even misclassification if the email topic does not fit into the previously identified categories for email topics.

References

Leevy, J.L., Khoshgoftaar, T.M., Bauder, R.A. *et al.* A survey on addressing high-class imbalance in big data. *J Big Data* **5**, 42 (2018).

<https://doi.org/10.1186/s40537-018-0151-6>

McCaney, K. (2012, September 26). The 20 most common words in phishing attacks. Route Fifty.

<https://www.route-fifty.com/cybersecurity/2012/09/the-20-most-common-words-in-phishing-attacks/280956/>

Pugh, R., & Wong, S. (2021, September 8). The top phishing keywords in the last 10k malicious emails we investigated. Expel. <https://expel.com/blog/top-phishing-keywords/>

Subhadeep Chakraborty. (2023). Phishing Email Detection [Data set]. Kaggle.

<https://doi.org/10.34740/KAGGLE/DSV/6090437>

Libraries

- [collections](#)
- [Datasets](#)
- [gc](#) (Garbage Collector)
- [joblib](#)
- [Matplotlib](#)
- [NumPy](#)
- [nltk](#) ([nltk.tokenize](#), [nltk.corpus](#), [nltk.stem](#))
- [pandas](#)
- [re](#)

- [seaborn](#)
- [sklearn.feature_extraction](#)
- [sklearn.model_selection](#)
- [sklearn.preprocessing](#)
- [sklearn.metrics](#)
- [string](#)
- [spicy.sparse](#)
- [torch](#) ([torch.cuda](#), [torch.device](#), [torch.utils.data](#))
- [Gradio](#)
- [transformers](#)
- [wordcloud](#)

Model APIs

- [bert-base-uncased](#)
- [roberta-base](#)
- [T5-small](#)
- [gensim](#) ([gensim.model.Ldamodel](#))
- [sklearn.feature_extraction.text.TfidfVectorizer](#)
- [sklearn.linear_model.LinearRegression](#)

Code

<https://github.com/caitlyn-p/Phishing-Email-Classification-and-Analysis>